

Project Scout

Alexandre Younes

Georgios Abboud

Advisor: Marcel Karam

12/19/2024

Doc version 1.0.0

Contents

1. Project Objective:	3
2. Project Scope:	3
2.1 In-Scope:	3
2.2 Out of Scope:	4
2.3 Limitations:	4
3. System Features	4
3.1 DOM Analysis Capabilities	4
3.2 AI Learning System	5
3.3 Navigation Assistance	6
4. Functional Requirements	6
4.1 DOM Analysis Requirements	6
4.2 AI Learning Requirements	7
4.3 User Interface Requirements	8
5. Non-Functional Requirements	8
6. System Flow	9
6.1 Initial Page Load	9
6.2 User Interaction Flow	10
6.3 Dynamic Content Handling	10
6.4 Learning Process	10
7. Project Implementation and Deployment Roadmap	10
8. Technology Stack	12
9. Research Papers that worked on a project related to ours.	14
9.1 Research papers that worked on a static web pages:	14
9.2 Research papers that worked on a dynamic web pages:	14
9.3 Research papers that worked on Task Understanding (Start and End Points)	15
9.4 Research papers that worked on Usability Gauging	15

1. Project Objective:

The project aims to develop an intelligent AI assistant that enhances web navigation by learning from user interactions. This assistant will analyze webpage structures in real-time, understand common navigation patterns, and provide clear guidance to users. Through DOM analysis and machine learning capabilities, the system will offer step-by-step instructions, helping users easily locate and interact with website elements. The assistant will continuously learn from user behaviors to improve its guidance accuracy and adapt to different website layouts. The goal is to create a seamless, user-friendly experience that makes web navigation more accessible and efficient for everyone.

2. Project Scope:

2.1 In-Scope:

1. Core AI Agent Features

- Real-time webpage DOM analysis (including dynamic content)
- Learning from user interactions
- Interactive guidance system
- Visual agent presence on webpages

2. Website Analysis

- Recognition of common web elements
- Page structure understanding
- Interactive element identification
- Navigation path analysis

3. AI Agent Interface

- Visual agent representation on webpage
- Direct interaction with users through simple messages
- Guide users through step-by-step actions

4. Learning System

- Continuous pattern recognition and adaptation
- User behavior analysis and learning

- Building knowledge base from all interactions

2.2 Out of Scope:

1. Website modification
2. Personal data processing
3. Form auto-filling

2.3 Limitations:

1. Performance might vary based on web application
2. Cannot access secure/encrypted webpage elements
3. Requires internet connection
4. Cross-Origin Restrictions (Cannot access DOM of websites from different origins)
5. Cannot inject our agent into third-party websites without permission
6. Captures element attributes and properties

3. System Features

3.1 DOM Analysis Capabilities

Feature	Description
Core Element Detection	a. Real-time scanning of webpage elements (both static and dynamic) b. Identifies interactive elements like buttons, forms, links automatically c. Tracks DOM changes and mutations as they happen d. Maps hierarchical relationships between elements e. Monitors element loading and rendering
Smart Element Analysis	- Extracts key properties (tagName, id, className, etc.) - Generates unique XPath identifiers for each element

	- Monitors element states (visibility, enabled/disabled, required) - Captures element attributes and properties
Dynamic Content Handling	- Detects and tracks AJAX-loaded content - Monitors real-time DOM mutations and changes - Analyzes asynchronously loaded elements - Tracks dynamic state changes - Handles single-page application content Updates - Maintains element references through DOM updates

3.2 AI Learning System

Feature	Description
Pattern Recognition	f. Learn common task completion paths g. Identify critical elements for tasks h. Understand task start/end points i. Map successful user journeys
Element Recognition	- Learn which elements are important for specific tasks - Identify interactive elements needed for completion - Understand element purpose in task context - Match elements to user goals
User Guidance	- Provide step-by-step task guidance - Highlight relevant elements - Give clear, contextual instructions - Guide users through task completion
Performance Analysis	- Track successful task completions - Monitor guidance effectiveness - Identify common user difficulties - Learn from successful interactions

3.3 Navigation Assistance

Feature	Description
Visual Presence	j. Clear visual indicator of AI agent k. Non-intrusive positioning on webpage l. Visible/hideable interface m. Status indicators (active/inactive)
Communication Interface	- Simple message display - Clear instructions to user - Element highlighting capability - Basic user feedback options
Interaction Methods	- Direct URL link provision when available - Simple element highlighting - Auto-scroll to target elements - Basic chat interface responses
Performance Analysis	- Track successful task completions - Identify common user difficulties - Learn from successful interactions

4. Functional Requirements

4.1 DOM Analysis Requirements

1. Element Detection

- R1.1: System shall detect all interactive elements (buttons, links, forms)
- R1.2: System shall identify element properties (id, class, type)
- R1.3: System shall generate unique element identifiers
- R1.4: System shall map element locations in DOM

2. Dynamic Content Handling

- R2.1: System shall detect AJAX content loading
- R2.2: System shall track DOM mutations
- R2.3: System shall maintain element references after updates
- R2.4: System shall handle single-page application changes

3. Structure Analysis

- R3.1: System shall analyze parent-child relationships
- R3.2: System shall identify main page sections
- R3.3: System shall map navigation hierarchies
- R3.4: System shall track element visibility states

4. Error Handling

- R4.1: System shall handle failed element detection gracefully
- R4.2: System shall recover from DOM analysis interruptions
- R4.3: System shall maintain stability during dynamic updates
- R4.4: System shall log analysis failures for debugging

4.2 AI Learning Requirements

1. Task Pattern Recognition

- R1.1: System shall identify successful navigation paths
- R1.2: System shall learn common task patterns
- R1.3: System shall store successful task completion data

2. Element Importance Learning

- R2.1: System shall identify critical elements for task completion
- R2.2: System shall learn which elements lead to successful outcomes
- R2.3: System shall recognize common interactive elements
- R2.4: System shall map elements to specific tasks

3. Guidance Learning

- R3.1: System shall improve guidance based on success rates
- R3.2: System shall learn effective navigation paths
- R3.3: System shall adapt to different webpage structures
- R3.4: System shall update guidance strategies based on user success

4. Data Collection

- R4.1: System shall collect task completion data
- R4.2: System shall track successful user paths
- R4.3: System shall store element interaction patterns

- R4.4: System shall maintain task success metrics

4.3 User Interface Requirements

1. Agent Visual Display

- R1.1: System shall display a visible AI agent on webpage
- R1.2: System shall position agent non-intrusively
- R1.3: System shall provide show/hide functionality
- R1.4: System shall indicate agent status (active/inactive)

2. Communication Display

- R2.1: System shall display clear guidance messages
- R2.2: System shall show instructions in readable format
- R2.3: System shall provide clear success/error feedback
- R2.4: System shall maintain message history

3. Element Interaction

- R3.1: System shall highlight relevant webpage elements
- R3.2: System shall provide direct links to target elements
- R3.3: System shall enable auto-scroll to target elements
- R3.4: System shall clear highlights after task completion

4. Accessibility

- R4.1: System shall ensure readable text size
- R4.2: System shall maintain sufficient contrast
- R4.3: System shall position interface accessibly
- R4.4: System shall not interfere with webpage functionality

5. Non-Functional Requirements

1. Performance Requirements

- NR1.1: System performance and analysis time shall scale proportionally with webpage size and complexity

- NR1.2: System must maintain UI responsiveness during analysis
- NR1.3: UI responses must occur within 100ms
- NR1.4: Element highlighting must be instantaneous

2. Reliability Requirements

- NR2.1: System must maintain 99% uptime
- NR2.2: System must recover automatically from DOM analysis errors
- NR2.3: System must work across different browsers (Chrome, Firefox, Safari)
- NR2.4: System must handle network interruptions gracefully

3. Resource Usage

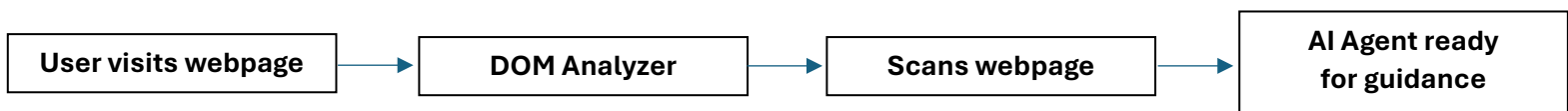
- NR3.1: CPU usage must not exceed 15% of browser resources
- NR3.2: Memory usage must not exceed 150MB
- NR3.3: Network requests must be limited to essential operations
- NR3.4: System must work on standard desktop/mobile devices

4. Security Requirements

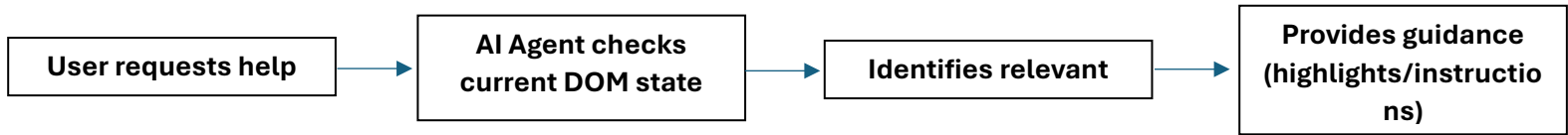
- NR4.1: System must not interfere with webpage security
- NR4.2: System must respect same-origin policies
- NR4.3: System must not store sensitive user data
- NR4.4: System must comply with website security policies

6. System Flow

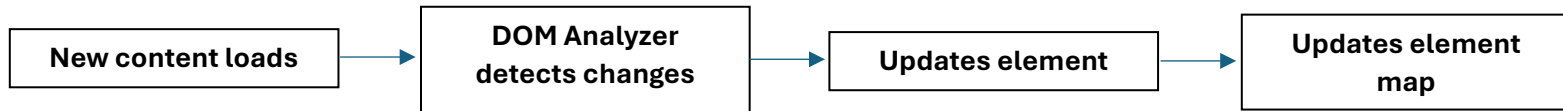
6.1 Initial Page Load



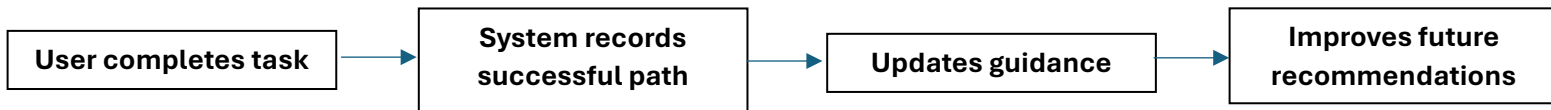
6.2 User Interaction Flow



6.3 Dynamic Content Handling



6.4 Learning Process



7. Project Implementation and Deployment Roadmap

1. Server Setup

Set up an Express.js server to handle incoming requests

2. DOM Analysis with Playwright

Utilize Playwright to automate browser actions and extract the DOM structure. Playwright supports multiple browser types, such as Chromium, Firefox, and WebKit.

3. DOM Extraction Logic

Launch a headless browser instance using Playwright, navigate to the specified URL, and extract the DOM structure. This involves evaluating the page's content and recursively extracting information about each element, such as tag names, class names, and text content.

4. Dynamic Content Handling

Implement a waiting mechanism to ensure dynamic content is captured. Wait for the dom content loaded event and add a short delay to account for asynchronous content loading.

5. API Endpoints for Analysis

Create an API endpoint (/api/analyze) that accepts a URL and returns the structured DOM data. Validate the URL, launch the browser, navigate to the page, and return the extracted DOM tree.

6. Interaction Recording and Data Export

Implement the Interaction Recorder component to capture user interactions on web pages. This component records events such as clicks, inputs, and form submissions. After recording, provide an option to export the interaction data as a CSV file. This CSV file will include detailed information about each interaction, such as event type, element details, and timestamps.

7. Data Preparation for Machine Learning

Use the exported CSV files to preprocess the data for machine learning. This involves cleaning the data, extracting relevant features, and structuring it in a format suitable for training. The CSV format facilitates easy integration with data analysis tools and machine learning frameworks.

8. Model Training

Train a machine learning model using the preprocessed data. The model will be designed to recognize patterns in the DOM structure and user interactions, enabling it to predict user behavior or automate tasks on similar web pages. This step involves selecting appropriate algorithms, tuning model parameters, and validating the model's performance.

9. Evaluation and Continuous Improvement

Evaluate the model's performance using a separate test dataset.

Analyze the results to identify areas for improvement. Continuously update the model with new data collected from user interactions to enhance its accuracy and capabilities. This iterative process ensures the model remains effective and relevant.

10. Agent User Interface Development

Develop a text-based interface for the AI agent. This interface should allow users to interact with the agent through text commands and queries. Include basic visual indicators, such as element highlights, to assist users in navigating the DOM.

11. Integration of AI Agent with DOM Visualizer

Integrate the AI agent with the DOM Visualizer to enable real-time guidance. The agent should provide text-based suggestions and highlight relevant elements within the visualizer to assist users in completing tasks.

12. Contextual Assistance and Recommendations

Implement the agent's ability to offer contextual assistance based on the DOM structure and user interactions. Use the trained model to analyze the current state and provide text-based recommendations to guide users effectively.

13. Feedback and Learning Mechanisms

Set up mechanisms for the agent to learn from user feedback and interactions. Capture user feedback on the agent's suggestions and use this data to refine the model, improving the agent's accuracy and relevance over time.

14. Scalability and Performance Optimization

Optimize the system for scalability to handle increased loads and more complex web pages. Refine backend processes and improve data processing efficiency to ensure the system can scale horizontally to accommodate more users and data.

15. Security and Privacy Measures

Implement robust security measures to protect user data and ensure privacy. Secure data transmission, anonymize user data, and comply with relevant data protection regulations. Conduct regular security audits and updates to maintain system integrity.

16. Advanced Analytics and Reporting

Develop analytics and reporting features to provide insights into user interactions and system performance. Create dashboards for monitoring key metrics and generating reports on user behavior patterns to identify areas for further improvement.

17. User Testing and Feedback Integration

Conduct user testing sessions to gather feedback on the system's usability and effectiveness. Use this feedback to make iterative improvements, ensuring the system meets user needs and expectations. Regular updates based on user feedback can help maintain user satisfaction and engagement.

Note: Due to cross-origin restrictions, the agent can only read the DOM of sites or pages that explicitly allow such access. This limitation ensures compliance with web security standards and respects the privacy settings of external websites.

8. Technology Stack

Backend Technologies

- **Node.js:** Used for building the server-side application, providing a scalable and efficient runtime environment for executing JavaScript code.
- **Express.js:** A web application framework for Node.js, used to set up the server and handle HTTP requests and responses.

- **Playwright:** Utilized for browser automation and DOM analysis, supporting multiple browser types like Chromium, Firefox, and WebKit.

Frontend Technologies

- **React:** A JavaScript library for building user interfaces, used to create dynamic and responsive components for the DOM Visualizer and Interaction Recorder.
- **Material-UI (MUI):** A React component library that provides pre-designed components, used to enhance the UI with a modern and consistent look.
- **React D3 Tree:** A React component for rendering tree structures, used to visualize the DOM tree in the frontend.

Data Handling and Communication

- **Axios:** A promise-based HTTP client for the browser and Node.js, used for making API requests from the frontend to the backend.
- **CSV:** Used for exporting interaction data, facilitating easy data manipulation and integration with machine learning tools.

Development and Deployment Tools

- **Nodemon:** A tool that automatically restarts the Node.js application when file changes are detected, used during development for efficiency.
- **dotenv:** A module that loads environment variables from a .env file, used to manage configuration settings securely.

AI and Machine Learning

- **Python:** Often used for machine learning tasks, though not explicitly mentioned in the current setup, it may be used for model training and data analysis.
- **TensorFlow/PyTorch:** Popular machine learning frameworks that could be used for training models on the collected interaction data.
- **Scikit-learn:** A machine learning library for Python, useful for data preprocessing and model evaluation.

Security and Middleware

- **CORS:** Middleware used to enable Cross-Origin Resource Sharing, allowing the frontend and backend to communicate securely across different origins.

9. Research Papers that worked on a project related to ours.

9.1 Research papers that worked on a static web pages:

1. Grounding Open-Domain Instructions to Automate Web Support Tasks

This study focuses on AI agents trained to interpret natural language instructions for automating tasks on static web pages.

<https://arxiv.org/abs/2103.16057>

2. Artificial intelligence (AI) for user experience (UX) design: a systematic literature review and future research agenda

This paper reviews AI methods aimed at optimizing static UX layouts and predicting user interaction patterns.

https://www.researchgate.net/publication/373389004_Artificial_intelligence_AI_for_user_experience_UX_design_a_systematic_literature_review_and_future_research_agenda

9.2 Research papers that worked on a dynamic web pages:

1. WebArena: A Realistic Web Environment for Building Autonomous Agents

This research introduces a platform for developing AI agents capable of interacting with dynamic web environments.

<https://arxiv.org/abs/2307.13854>

2. Generative AI at Work

This study explores the use of conversational AI tools that assist users in dynamic customer service scenarios.

3. AI Customer Service: Task Complexity and Problem-Solving Ability

This paper examines AI applications in customer interactions on dynamic platforms, focusing on varying task complexities.

https://www.researchgate.net/publication/341561736_AI_Customer_Service_Task_Complexity_Problem-Solving_Ability_and_Usage_Intention

4. Conversational AI and Chatbots: Enhancing User Experience on Websites

This research explores how conversational AI and chatbots improve user engagement and satisfaction on dynamic websites.

https://www.researchgate.net/publication/383035963_Conversational_AI_and_Chatbots_Enhancing_User_Experience_on_Websites

9.3 Research papers that worked on Task Understanding (Start and End Points)

1. Collaboration with Conversational AI Assistants for UX Evaluation

This study investigates how AI conversational agents identify tasks through real-time interaction, establishing clear start and end markers for analysis.

<https://arxiv.org/abs/2303.03638>

2. Proactive and Reactive Help from Intelligent Agents

This paper discusses AI agents' ability to predictively identify user tasks and proactively guide users, marking task completion explicitly.

3. When Do Humans Heed AI Agents' Advice? When Should They?

This research describes experiments where AI assists users in decision-making tasks, defining task start and end through interactive advice sessions.

<https://pmc.ncbi.nlm.nih.gov/articles/PMC11089830/>

4. Identifying User Goals from UI Trajectories

This paper introduces a method for inferring user intentions based on their interactions with graphical user interfaces, aiding AI agents in understanding task boundaries.

<https://arxiv.org/abs/2406.14314>

9.4 Research papers that worked on Usability Gauging

1. The Impact of AI on UX: Challenges and Opportunities

This study evaluates AI's impact on web usability by analyzing pre- and post-deployment user behaviors and feedback.

2. AI for UX Research: How Artificial Intelligence is Transforming User Experience Studies

This paper discusses AI's role in automating usability testing, highlighting metrics like task success rates and user satisfaction scores.

3. AI-Powered Tools for UX Research: Issues and Limitations

This research reviews AI-based tools for usability analysis, focusing on post-deployment feedback mechanisms.

<https://www.nngroup.com/articles/ai-powered-tools-limitations/>

4. User-Centric AI: Evaluating the Usability of Generative AI Applications Through User Reviews on App Stores

This article presents a usability evaluation of generative AI applications by analyzing user reviews from digital marketplaces.

https://www.researchgate.net/publication/385261015_User-centric_AI_evaluating_the_usability_of_generative_AI_applications_through_user_reviews_on_app_stores

5. **Enhancing User Acceptance of an AI Agent's Recommendation in Decision-Making Processes**

This study introduces a methodology to optimize user interfaces for effectively conveying AI-based agent outputs to users, aiming to achieve high acceptance rates of recommendations.

<https://www.mdpi.com/2076-3417/14/17/7874>

6. **Usability Evaluation and Enhancement of the AI-Powered Smart-Campus Framework: A User-Centred Approach**

This study employs a user-centered approach to improve user experience and maximize system functionality in an AI-powered smart-campus framework.

7. **UX Design Considerations for Human-AI Agent Interaction**

This project explores the challenges and design requirements involved in developing safe and user-centered experiences for autonomous AI agents.

<https://cltc.berkeley.edu/publication/ux-design-considerations-for-human-ai-agent-interaction/>

8. **Enhancing UX Evaluation Through Collaboration with Conversational AI Assistants**

This work examines how AI can assist UX evaluators in analyzing usability tests through interactive conversational assistants.

9. **Enhancing Usability and Control in Artificial Intelligence of Things (AIoT) Environments Through Semantic Web Control Models**

This chapter introduces Semantic Web control models, leveraging technologies like RDF, OWL, and SPARQL, to enhance usability in AIoT environments.

https://www.researchgate.net/publication/380656774_Enhancing_Usability_and_Control_in_Artificial_Intelligence_of_Things_Environments_AIoT_Through_Semantic_Web_Control_Models

Our system introduces a small but significant advancement (5–15%) over existing research by addressing gaps in prior work across several key areas. While existing studies on static pages focus on predefined workflows and lack adaptability, our system extends real-time, context-aware assistance to both static and dynamic environments, ensuring broader applicability.

Compared to dynamic page solutions like "WebArena," which prioritize agent autonomy, our system integrates dynamic task assistance with continuous usability feedback loops, enabling iterative design improvements. Unlike task understanding approaches that rely on rigid, predefined markers, our system uses behavioral and contextual cues to intuitively identify task start and end points, offering seamless and adaptive guidance.

Existing usability evaluation methods often focus on retrospective analysis post-deployment, but our system shifts to a proactive approach by collecting real-time user interaction data to enable continuous usability enhancements during task execution. Moreover, by combining task guidance and usability analysis into a single, unified framework, our system bridges the gap between immediate user assistance and long-term design optimization. This dual functionality, coupled with dynamic adaptability and continuous learning, positions our AI-driven help agent as a more versatile, user-centric solution that evolves alongside user needs and addresses the limitations of existing research.